**Name & Surname:** Musawakhe Chipeya **St number:** 45550654
**Name & Surname:** Asemahle Maqizana **St number:** 37485299

Please use this document to answer the questions.

---

**GitHub repository (public for marking):** https://github.com/Asemahle2244/phishing-email-detection-model · Project: Phishing email detection with a BiLSTM, served through a FastAPI endpoint and a Tkinter desktop app.

---

# 3 Theory component (30 marks)

## 3.1 Model scenario (3 marks)

*Give the scenario of your model. How is this model useful in the context of cybersecurity?*

Phishing is one of the most common ways attackers first get into an organisation. A single convincing email can lead to a stolen password or a payment to a fraudster, or it can deliver malware. Our model is a **phishing email classifier** that sits between the mail server and the user's inbox. It reads the subject and body of each incoming email and returns three things:

- the probability that the email is phishing or fraud,
- a decision (*phishing* or *legitimate*) based on a tuned threshold,
- the words that most influenced that decision.

In practice it would be used in two ways:

1. **As an automated filter at the email gateway.** Emails above the threshold are quarantined or tagged with a warning banner before the user sees them.
2. **As a triage tool for a security analyst or help-desk agent.** A user forwards a suspicious email, and the analyst pastes it into our desktop application. The analyst gets a verdict in about 25 ms, together with the highlighted words that explain it, so the decision can be checked rather than blindly trusted.

The model reduces the number of phishing emails that reach users and cuts the time analysts spend on manual triage. The highlighted explanation also helps train users to recognise the warning signs themselves. Examples of those signs are urgency ("within 24 hours"), account threats ("suspended", "limited") and requests to "verify" credentials through a link.

## 3.2 Model architecture (10 marks)

*Explain all relevant details of the model you used. For example, if you implemented a Convolutional Neural Network (CNN), specify the types of layers, the activation functions used, and the overall model architecture. Also explain why the chosen model is well suited to the data it was trained on.*

The main model is a **Bidirectional Long Short-Term Memory (BiLSTM)** recurrent neural network built in PyTorch. A TF-IDF + Logistic Regression model was also trained as a classical baseline for comparison.

### Input pre-processing

- The subject and body are joined, HTML tags are removed and the text is lower-cased.
- URLs, email addresses and numbers are replaced with the placeholder tokens `urltoken`, `emailtoken` and `numtoken`. The model therefore learns that a link is present instead of memorising specific domains, which change constantly.

- The characters `!` `?` `$` `%` are kept as separate tokens, because urgency and money symbols are useful signals.
- The text is split into words. A vocabulary of the 25,000 most frequent words is built from the **training set only**, and unknown words map to `<unk>`.
- Each email is truncated to its first 256 tokens.

### Layers

| # | Layer | Configuration | Output shape | Activation |
|---|---|---|---|---|
| 1 | Input | token ids, max length 256 | (B, 256) | – |
| 2 | Embedding | 25,000 × 100, learned from scratch | (B, 256, 100) | – |
| 3 | Dropout | p = 0.3 | same | – |
| 4 | Bidirectional LSTM | 1 layer, 64 units per direction, packed sequences | (B, 256, 128) | sigmoid gates, tanh cell/hidden |
| 5 | Masked mean + max pooling | concatenated over time | (B, 256) | – |
| 6 | Fully connected | 256 → 64 | (B, 64) | ReLU |
| 7 | Dropout | p = 0.3 | same | – |
| 8 | Output | 64 → 1 logit | (B,) | sigmoid → P(phishing) |

The model has 2,601,505 trainable parameters. Of these, 2.5 million are in the embedding layer.

**Embedding layer.** Each word id becomes a dense 100-dimensional vector. These vectors are learned during training, so words used in similar contexts (for example "verify" and "confirm") end up close together.

**LSTM.** An LSTM (Hochreiter & Schmidhuber, 1997) reads the email one token at a time and keeps a memory cell. Three gates control what is stored, forgotten and output: the input, forget and output gates, each with a sigmoid activation. The candidate memory and hidden state use tanh. This gating avoids the vanishing-gradient problem of simple RNNs, so the model can link words that are far apart, such as "your account" early in an email and "click here to verify" much later.

**Bidirectional.** A second LSTM reads the sequence in reverse (Schuster & Paliwal, 1997). Each position therefore has context from both directions, and the two 64-unit outputs are concatenated into 128 values. Packed sequences ensure the padding added to short emails is ignored.

**Pooling.** Two kinds of pooling are applied over all positions. *Max-pooling* captures the single strongest phishing signal anywhere in the email. *Mean-pooling* summarises its overall tone. A mask excludes padding from both.

**Classifier head.** A 64-neuron fully connected layer with ReLU, then dropout and a single output neuron. A sigmoid turns the output into the probability of phishing.

### Training set-up

- **Loss:** binary cross-entropy with logits. A **positive-class weight of 4.95** compensates for there being about five legitimate emails per phishing email.
- **Optimiser:** Adam (Kingma & Ba, 2015) with a learning rate of 0.001 and a batch size of 64. Gradient-norm clipping at 1.0 prevents exploding gradients.
- **Batching:** emails of similar length are batched together (length bucketing) to reduce padding.
- **Regularisation:** dropout of 0.3 (Srivastava et al., 2014) and **early stopping** with a patience of 3 epochs on validation loss (Prechelt, 1998). The best weights are restored when training stops.

- **Decision threshold:** tuned on the validation set to maximise F1, giving 0.695.

### Why this architecture suits the data

Phishing is a *language* problem. Meaning depends on word order and context: "verify your account" is suspicious, while "your account was verified" usually is not. A bag-of-words model loses that order. The BiLSTM keeps it and sees context on both sides of every word.

Phishing emails also vary greatly in length, and the persuasive part can sit anywhere in the message. Recurrence with max-pooling handles variable length and finds the key phrase wherever it occurs.

Finally, the model is small enough (2.6 million parameters) to train on a laptop CPU in about 11 minutes. That makes retraining practical when new phishing styles appear.

## 3.3 Model result analysis (10 marks)

*Explain the results of your model using the metrics and graphs derived from the model's training. Is this model effective in the problem it is trying to resolve?*

### Dataset and evaluation set-up

We used the Kaggle *Phishing Email Dataset* collection (Al-Subaiey et al., 2024).

- **Phishing (1):** `Nazario.csv`, 1,565 real phishing emails from 2015–2022 (including fake Standard Bank and Nedbank emails), and `Nigerian_Fraud.csv`, 3,332 advance-fee scam emails.
- **Legitimate (0):** the legitimate emails from `CEAS_08`, `SpamAssassin` and `Ling`.
- After removing 333 exact duplicates, 28,368 unique emails remained.
- The data was split into stratified **training / validation / test sets of 70 / 15 / 15** (19,857 / 4,255 / 4,256). The phishing share is 16.8% in each set. A code check confirmed that no email appears in more than one set.

### Training behaviour (loss and metric curves)

- **Training loss** fell steadily, from 0.363 in epoch 1 to 0.012 in epoch 7.
- **Validation loss** reached its minimum of 0.0269 at **epoch 4**, then rose to 0.030, 0.034 and 0.037. This is the start of overfitting: the model kept improving on data it had seen while getting slightly worse on data it had not.
- **Early stopping** ended training after epoch 7 (patience 3) and restored the epoch-4 weights.
- **Validation F1** stayed between 0.97 and 0.98 from epoch 3 onwards. The model learned the task quickly, and most of the later epochs only refined its confidence.

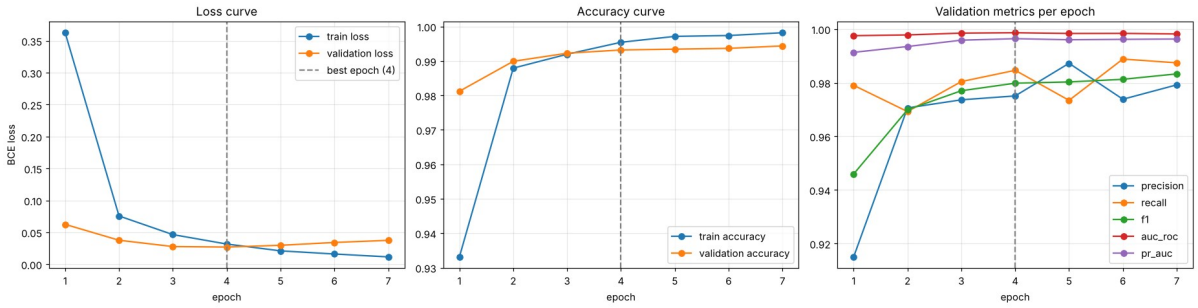



*Figure 1: BiLSTM training curves per epoch: loss (left), accuracy (centre) and validation metrics (right). The dashed line marks the best epoch (4), whose weights were restored by early stopping.*

## Test-set results (4,256 unseen emails)

| Model | Accuracy | Precision | Recall | F1 | AUC-ROC | PR-AUC |
|---|---|---|---|---|---|---|
| TF-IDF + Logistic Regression (baseline) | 0.9946 | 0.9929 | 0.9748 | 0.9838 | 0.9996 | 0.9987 |
| BiLSTM (threshold 0.5) | 0.9941 | 0.9739 | 0.9916 | 0.9827 | 0.9996 | 0.9986 |
| BiLSTM (tuned threshold 0.695) | 0.9955 | 0.9833 | 0.9902 | 0.9868 | 0.9996 | 0.9986 |

- **Confusion matrix (BiLSTM):** 708 phishing emails were caught and 3,529 legitimate emails were correctly passed. There were **12 false positives** (0.34% of legitimate mail flagged) and **7 false negatives** (0.98% of phishing missed).

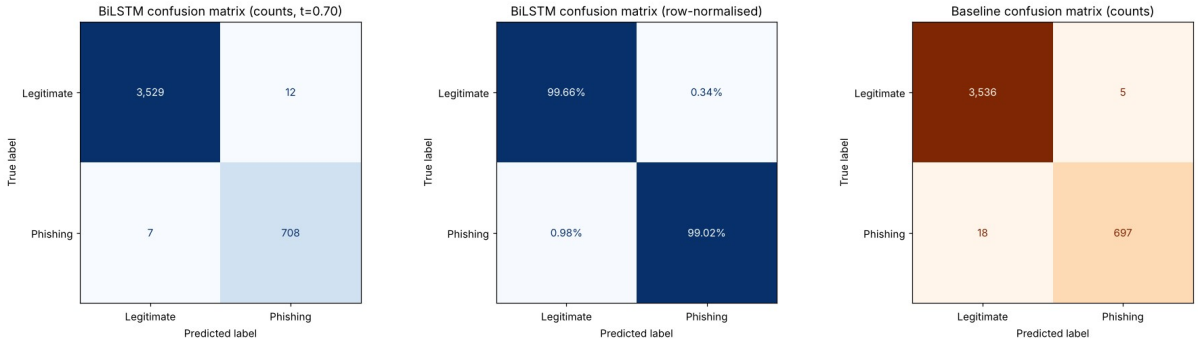



*Figure 2: Confusion matrices on the test set: BiLSTM counts (left), BiLSTM row-normalised (centre) and the TF-IDF baseline (right).*

- **ROC and PR curves:** both models reach an AUC of about 0.9996. The PR-AUC of 0.9986 is the more informative figure here, because it focuses on the minority phishing class, and it confirms that the high scores are not an artefact of class imbalance. (With 16.8% phishing, a model that always answered "legitimate" would score 83% accuracy but zero recall. This is why we report precision, recall, F1 and PR-AUC rather than accuracy alone.)

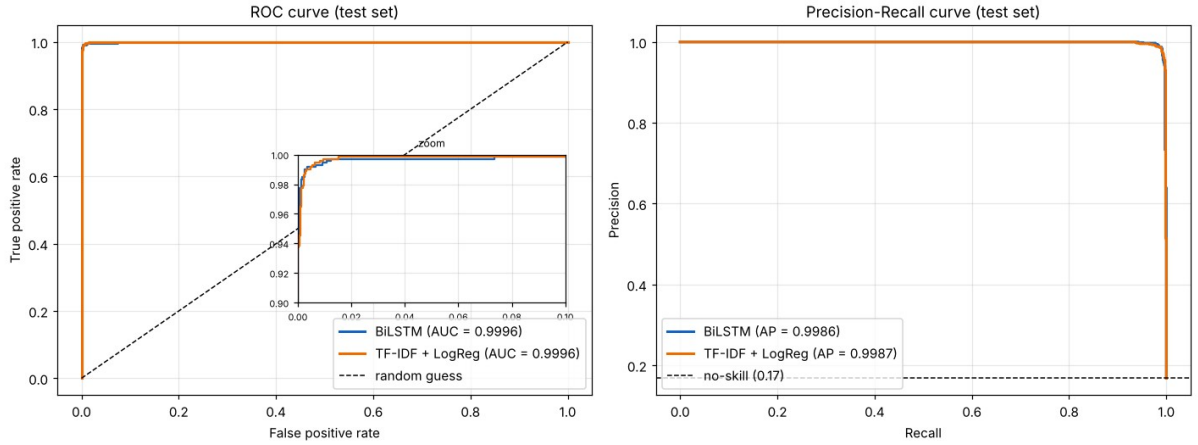



*Figure 3: ROC curve (left, with a zoomed inset) and precision–recall curve (right) for both models on the test set.*

- **BiLSTM versus baseline:** the BiLSTM has the best F1 and accuracy and a clearly higher **recall** (99.0% against 97.5%). It missed 7 phishing emails against the baseline's 18, fewer than half as many, and missed phishing matters most in security. The baseline has slightly higher precision. The gap is small, which shows that many phishing emails carry strong single-word cues. The BiLSTM gains on emails where the context and word order carry the meaning.

- **Per-source check:** test accuracy is above 98.5% on every source: CEAS 99.6%, Ling 99.2%, Nazario 98.6%, Nigerian Fraud 99.2% and SpamAssassin 100%. This shows the model is not just recognising one dataset.

### Error analysis

- **False negatives** include: a voicemail-notification phish; an "Email Quota" phish with a single link; a short "payment confirmation, see attachment" lure whose malicious part was the attachment, which the model cannot see; and two Nazario emails that are really spam (a Viagra advert and a paid-survey offer). Those two show **label noise** in the source data.
- **False positives** include: a user asking for help with a confirmation link; a survey invitation offering "$10"; and bulk mailing-list mail. These look like phishing on the surface.

These errors show the limits of a text-only model. It has no information about attachments, sender reputation or the real destination of links.

### Effect of defining the problem correctly

Our **first version** was trained on CEAS_08 alone, with spam as the positive class. It reached 99.3% test accuracy, yet it scored a basic PayPal credential-phishing email as only **0.4%** likely to be phishing. Its explanations showed why: the word "account" pushed strongly towards *legitimate*, because in CEAS the word mostly appears in IT mailing lists.

After retraining on real phishing, the same email scores **100%**, and new hand-written South African examples (an FNB banking lure, a Microsoft 365 password-expiry lure) score above 99%.

The lesson is that a high test score only shows the model fits *its own* data. Whether it solves the real problem depends on whether that data represents the threat.

### Is the model effective?

Yes, for the problem it was trained on. It catches 99% of phishing in held-out data, while flagging fewer than 1 in 250 legitimate emails, and it explains each decision.

The results are probably **optimistic** for live deployment, for three reasons:

- the phishing emails are historical (up to 2022);
- the legitimate mail comes from older public corpora;
- phishing and legitimate emails come from different collections, which may make them easier to tell apart than real mixed inbox traffic.

We reduced this risk by:

- masking the Nazario collector's name and address, which appeared in almost every phishing email;
- using legitimate mail from three corpora;
- checking accuracy per source.

It still remains the main limitation.

## 3.4 Real-world use of models (7 marks)

*Evaluate the real-world use of models within the field of cybersecurity, focusing on how these models are applied to tasks such as intrusion detection, threat classification, anomaly detection, and risk assessment. Discuss their effectiveness in practical deployments, including strengths, limitations, and common challenges such as data quality, concept drift, and false positives. Use academic references.*

Machine-learning models are now widely used in security operations.

- **Intrusion detection:** models classify network flows or host events as normal or malicious (Buczak & Guven, 2016).
- **Threat classification:** models sort malware samples and phishing messages into families.
- **Anomaly detection:** a model learns "normal" behaviour, such as login times or data volumes, and flags deviations.
- **Risk assessment:** model scores are combined with asset value to prioritise alerts.

Email security gateways and endpoint products commonly combine supervised classifiers like ours with sender reputation, URL sandboxing and attachment analysis.

### Strengths in practice

- **Scale and speed:** a trained model classifies thousands of emails or flows per second, far beyond human capacity.
- **Generalisation beyond signatures:** unlike rule- or signature-based tools, ML can recognise *new* variants that share the patterns of known attacks. Deep models such as RNNs and CNNs have reported high accuracy on phishing email detection (Fang et al., 2019; Al-Subaiey et al., 2024).
- **Consistency and triage:** scores let analysts focus on the riskiest cases first.
- **Explainability:** methods such as LIME (Ribeiro et al., 2016), and the occlusion explanations used in our application, make decisions auditable and help build trust with analysts.

### Limitations and common challenges

- **Data quality and representativeness.** Models are only as good as their training data. Public security datasets are often old, labelled imperfectly and unrepresentative of real traffic (Sommer & Paxson, 2010; Engelen et al., 2021). Arp et al. (2022) list sampling bias, label inaccuracy and data snooping among the most common pitfalls in security ML. We saw several of these directly:
  - spam labelled as phishing in our first dataset;
  - label noise in Nazario;
  - collection artefacts that a model could exploit as shortcuts.
- **Concept drift.** Attackers change tactics continuously, so the statistical patterns a model learned slowly stop matching reality (Gama et al., 2014). Pendlebury et al. (2019) showed that malware classifiers evaluated on randomly split data perform much worse when tested on *later* samples. A random train/test split like ours therefore overestimates future performance. AI-generated phishing, which is fluent and free of the usual spelling mistakes, is a current example of drift. Deployed models need ongoing monitoring and regular retraining.
- **False positives and the base-rate fallacy.** Malicious events are rare compared with legitimate ones. Even a small false-positive rate therefore produces many false alarms in absolute numbers (Axelsson, 2000). For example, a 0.34% false-positive rate on 100,000 legitimate emails a day is still about 340 wrongly flagged emails a day. Too many alerts cause **alert fatigue**, and analysts start ignoring them. This is why thresholds must be tuned to the organisation's cost of a missed attack versus a false alarm.
- **Adversarial evasion.** Attackers can deliberately craft inputs to evade a known model, for example by misspelling trigger words, hiding text in images or adding benign padding (Biggio & Roli, 2018). Text-only models like ours are vulnerable to this.
- **Limited view.** A single model sees only part of the picture. Ours cannot inspect attachments, links or sender history. In practice, ML is most effective as **one layer** in defence-in-depth, combined with technical controls (SPF, DKIM, DMARC, URL sandboxing) and user awareness training.

### Conclusion

ML models are effective and already essential in cybersecurity because they scale and can generalise to new attacks. Their real-world value depends far less on headline accuracy than on:

- representative, well-labelled data;
- monitoring for drift;
- managing false positives;
- explainable output that analysts can verify.

Our project showed this directly. A model with over 99% accuracy can still fail at the real task if the data does not represent the threat.

## References


- Al-Subaiey, A., Al-Thani, M., Alam, N.A., Antora, K.F., Khandakar, A. & Zaman, S.A.U. 2024. Novel interpretable and robust web-based AI platform for phishing email detection. arXiv preprint arXiv:2405.11619.
- Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L. & Rieck, K. 2022. Dos and don'ts of machine learning in computer security. *Proceedings of the 31st USENIX Security Symposium*. pp. 3971–3988.
- Axelsson, S. 2000. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security*, 3(3):186–205.
- Biggio, B. & Roli, F. 2018. Wild patterns: ten years after the rise of adversarial machine learning. *Pattern Recognition*, 84:317–331.
- Buczak, A.L. & Guven, E. 2016. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176.
- Engelen, G., Rimmer, V. & Joosen, W. 2021. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. *IEEE Security and Privacy Workshops (SPW)*. pp. 7–12.
- Fang, Y., Zhang, C., Huang, C., Liu, L. & Yang, Y. 2019. Phishing email detection using improved RCNN model with multilevel vectors and attention mechanism. *IEEE Access*, 7:56329–56340.
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. & Bouchachia, A. 2014. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):Article 44.
- Hochreiter, S. & Schmidhuber, J. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Kingma, D.P. & Ba, J. 2015. Adam: a method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.
- Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J. & Cavallaro, L. 2019. TESSERACT: eliminating experimental bias in malware classification across space and time. *Proceedings of the 28th USENIX Security Symposium*. pp. 729–746.
- Prechelt, L. 1998. Early stopping – but when? (*In* Orr, G.B. & Müller, K.-R., *eds*. Neural networks: tricks of the trade. Berlin: Springer. pp. 55–69).
- Ribeiro, M.T., Singh, S. & Guestrin, C. 2016. "Why should I trust you?": explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. pp. 1135–1144.
- Schuster, M. & Paliwal, K.K. 1997. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681.
- Sommer, R. & Paxson, V. 2010. Outside the closed world: on using machine learning for network intrusion detection. *IEEE Symposium on Security and Privacy*. pp. 305–316.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. 2014. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958.